

# Optimal Strategies for the WordHunt Oracle Game

A research logbook (Volume 2)

Started 2026-09-18

This document is self-contained: it states the game, the model, and the objective from scratch, and assumes no prior context. It supersedes an earlier volume (`archive/logbook.v1_getting_started.tex`) which recorded the process of arriving at a correct model and is kept only as a historical record – nothing in it should be taken as current. Every claim and number here has been independently verified against brute-force enumeration or an independently-coded simulation, as described alongside it; none is carried over unverified from the earlier volume.

## 1 The game

A grid of  $n \times n$  cells is filled with distinct letters from an alphabet of  $n^2 + 1$  symbols, so that exactly one letter of the alphabet never appears anywhere in the grid. (The real game is  $n = 5$ : a 26-letter alphabet, one letter absent, a  $5 \times 5$  grid.)

A **line** is any of the  $n$  rows,  $n$  columns, or 2 main diagonals –  $2n + 2$  lines in total, each of length  $n$ . The grid is constructed so that reading the letters along *exactly one* line spells a word from a fixed vocabulary; every other line's letters do not spell a vocabulary word. This one-and-only-one-word property is a defining guarantee of the real game, not an assumption we are choosing to make – a grid violating it is never published.

The player is told, before play starts, which single letter is absent from the alphabet. Play then proceeds in turns: each turn, the player names an untested letter, and the oracle discloses *the one cell where that letter actually sits in the grid* – regardless of whether that letter turns out to be part of the answer word or not. (The grid is a bijection between the  $n^2$  occupied cells and the  $n^2$  non-absent letters, so every query has a definite, single-cell answer; there is no "that letter isn't part of the word" response with no location attached.)

The game ends the instant every cell of the answer line has been individually disclosed this way – *not* at the moment the player could, in principle, deduce the answer from partial information. If the answer is logically forced with two of its letters still untested, the player must still spend two more turns naming exactly those two letters (each of which will, of course, turn out to be a hit) before the oracle recognises the line is complete and the game reports how many turns it took. This is the actual scoring rule of the real game, and it is the central fact this document's objective (Section 3) is built around.

## 2 The generative model

To reason about optimal play we need an explicit probabilistic model of how a grid satisfying Section 1's guarantee comes to exist. We use the simplest one consistent with that guarantee, and

call it  $M_2$  (the label is kept from earlier internal work so this document can be extended later without renumbering; no other model appears in this document).

$M_2$  generates a grid as:

1. word  $w \sim \text{Uniform}(\text{vocabulary } W)$
2. line  $p \sim \text{Uniform}(\text{the } 2n + 2 \text{ lines})$
3. excluded letter  $e \sim \text{Uniform}(\text{alphabet} \setminus w)$
4. the remaining  $n^2 - n$  letters are placed in the remaining  $n^2 - n$  cells via a uniformly random permutation
5. **reject and resample from step 1** if any line other than  $p$  also happens to spell a vocabulary word

Step 5 is what makes  $M_2$  consistent with Section 1’s guarantee: without it, a grid can accidentally spell a second word by coincidence, and it is not merely rare – Section 5 measures this directly and finds it common enough, for realistic vocabulary sizes, to matter.

The excluded letter  $e$  is revealed to the player before play starts (as in the real game, Section 1), so it is fixed per game instance and never treated as a live hypothesis in what follows.

**Scope.**  $M_2$ ’s vocabulary  $W$  is a small, fixed, hand-picked set of length- $n$  strings for the board sizes studied here ( $n = 2, 3$ ) – deliberately not English, just a toy dictionary large enough to be non-trivial. Real English vocabularies (e.g. the  $n = 5$  game’s `dict_game.csv`, 10,176 words) are discussed only as a scale reference in Section 7; no result in this document has yet been computed against one under a verified solver.

### 3 The objective

Let  $T$  be the number of turns played until the game ends, in the sense of Section 1: the smallest  $t$  such that every cell of the true answer line has been disclosed by one of the first  $t$  queries. The objective is to choose, adaptively (informed by every prior query and its outcome), the sequence of letters to query so as to minimise  $\mathbb{E}[T]$  under  $M_2$ .

**A useful related quantity: certainty.** Separately from  $T$ , define  $C \leq T$  as the turn at which the surviving hypothesis space – every  $(w, p)$  pair still consistent with everything disclosed so far – first collapses to a single pair. Under  $M_2$ ’s guarantee (unlike a model without it), this is always well-defined and unambiguous: two distinct  $(w, p)$  pairs can never remain simultaneously consistent with a fully-revealed grid, because the grid cannot spell two words.  $C$  is not what the game scores, and minimising  $\mathbb{E}[C]$  is a different optimisation problem from minimising  $\mathbb{E}[T]$  – Section 5 shows the two optimal policies genuinely disagree, not just at their optimal values but at individual decision points throughout the game.  $C$  is retained here because it is analytically convenient, always  $\leq T$ , and useful as a baseline for measuring how much  $T$ ’s ”mandatory mop-up” after certainty actually costs.

### 4 Exact solution method

**State.** The relevant state for computing  $\mathbb{E}[T]$  optimally is the pair  $(H, R)$ :  $H$ , the set of  $(w, p)$  hypotheses still consistent with everything disclosed so far, and  $R$ , the set of grid cells disclosed so far. Both are needed –  $R$  is not recoverable from  $H$  alone, because two different query histories

can leave behind the same surviving hypothesis set while having disclosed different amounts of whichever line turns out to be the true one, and that difference changes  $T$  directly. A query landing on a true-line cell does double duty: it can narrow  $H$  and it always advances the mandatory reveal: a query landing on a dross cell only narrows  $H$  (if it discriminates between hypotheses at all).

**Recursion.** Write  $A \subseteq H$  for the hypotheses whose own line is not yet fully covered by  $R$  (an hypothesis whose line *is* fully covered has already "finished" – if it were secretly true, the game would already have ended):

$$V(H, R) = 0 \quad \text{if } A = \emptyset \quad (1)$$

$$V(H, R) = \frac{|A|}{|H|} \left[ 1 + \min_L \sum_c \frac{|A_c|}{|A|} V(A_c, R \cup \{c\}) \right] \quad \text{otherwise} \quad (2)$$

where  $A_c \subseteq A$  is the subset of active hypotheses in which letter  $L$  actually sits at cell  $c$ , and the minimisation is over untested letters.  $\mathbb{E}[T] = V(H_0, \emptyset)$  for the initial hypothesis set  $H_0$ . A letter is never excluded from the search merely for failing to discriminate between hypotheses in  $A$ : it can still be valuable purely by advancing the mandatory reveal for some or all of them, and this objective must credit that. (A letter whose cell is *already* in  $R$  must be excluded, though – not merely discouraged – since considering it recurses on the identical state before that state is cached, which is an infinite loop, not just a wasted turn.)

An analogous, simpler recursion computes  $\mathbb{E}[C]$ : terminate when  $H$  collapses to one  $(w, p)$  pair rather than checking per-hypothesis line coverage, with no need to track  $R$  as separate state.

**Implementation.** `solver.py`: `MopUpSolver` computes  $V(H, R)$  (the real objective); `TagIdentifySolver` computes the certainty analogue. Both are memoised expectimax searches over exhaustively enumerated worlds (`enumerate_worlds`, with  $M_2$ 's step 5 veto implemented as a direct rejection filter on the enumerated set). Every numeric claim below was independently cross-checked by at least one of: an independently-coded re-derivation of the same recursion (`simulate_mopup_policy_exact_expectation`), or literal step-by-step play against every enumerated world as ground truth with no expectation formula at all (`simulate_mopup_bruteforce`) – `tests/test_mopup_solver.py` runs both at  $n = 2$  for every result in Section 5 and passes.

**Greedy comparator.** A natural one-step heuristic – at each turn, query the letter that maximises the Shannon entropy of the immediate outcome partition, ignoring everything beyond that one query – is implemented as `greedy_choice`. It is a proxy, not a claim of optimality: Section 5 measures exactly how much it costs to follow it instead of the true optimum.

## 5 Results at $n = 2$

Primary vocabulary (alphabet  $\{a, b, c, d, e\}$ ):  $W = \{\text{ab, bc, cd, de, ea}\}$ , a 5-word cycle.

### 5.1 How often does $M_2$ 's veto actually trigger?

Measured directly, as the fraction of  $(\text{word}, \text{line}, \text{excluded})$  combinations for which *no* arrangement of the remaining letters avoids a second word appearing (i.e. step 5 above always rejects):

| vocabulary                                             | words | impossible combinations |
|--------------------------------------------------------|-------|-------------------------|
| sparse ( $\{\text{ab, cd, be}\}$ , minimal overlap)    | 3     | 7.4%                    |
| primary ( $\{\text{ab, bc, cd, de, ea}\}$ , a 5-cycle) | 5     | 44.0%                   |
| denser ( $\{\text{ab, cd, ae, bc}\}$ )                 | 4     | 36.1%                   |
| degenerate (every possible string counts as "a word")  | 20    | 100.0%                  |

The 100% figure is not a bug: if literally every string is a vocabulary word, every line of every grid trivially "spells a word", so no grid can ever satisfy the one-word guarantee –  $M_2$  is unsatisfiable by construction for that vocabulary, and is correctly reported as such rather than crashing or returning something meaningless. The rate is driven by vocabulary density (how much letter-overlap structure creates opportunities for an accidental second word), not by anything specific to  $n = 2$ 's geometry – confirmed by measurement across four vocabularies spanning the full range from near-zero to total impossibility, rather than assumed from the single primary case.

## 5.2 Optimal values

For the primary vocabulary, at every excluded-letter scenario (all 5 give the same values by this vocabulary's cyclic symmetry):

| quantity                         | optimal value | greedy value         |
|----------------------------------|---------------|----------------------|
| $\mathbb{E}[C]$ (certainty)      | 2.0909        | 2.0909               |
| $\mathbb{E}[T]$ (real objective) | 2.2727        | <b>2.5455–3.4545</b> |

Two things worth separating out:

- **The mandatory mop-up tax:**  $\mathbb{E}[T] - \mathbb{E}[C] = 0.1818$  queries under optimal play, exactly, in every scenario for this vocabulary – the measurable cost of the real scoring rule over merely reaching certainty.
- **Greedy is optimal for  $C$  but not for  $T$ , on this vocabulary.** One-step maximum-entropy selection happens to match the true optimum for the certainty objective (2.0909 in both columns), but is measurably worse for the real objective – greedy's value ranges up to 3.4545 depending on excluded letter, against the true optimum of 2.2727 throughout. Greedy does not see the double-duty value of a query landing on the true line, and a policy tuned only for reaching certainty fastest can leave real mop-up queries on the table.

## 5.3 The optimal policies for $C$ and $T$ genuinely disagree

Method: solve for  $\mathbb{E}[T]$  exhaustively (the solver's own search tries every letter at every reachable state, so its memoised results cover the whole reachable decision tree, not just the states visited along its own optimal path). At every state where *both* the  $T$ -optimal and  $C$ -optimal solvers still faced a genuine choice (states where the  $C$ -optimal solver had already reached certainty, and so had nothing left to decide, are not a meaningful comparison – that is exactly the point of the mop-up tax above), compare their chosen letters, and where they differ, compute the exact  $T$ -cost of forcing the  $C$ -optimal choice instead, to separate real disagreement from harmless ties (multiple equally-good letters).

Exhaustive across all 5 excluded-letter scenarios: 135 states where both solvers faced a real choice; **65 disagreed (48%)**, and **all 65 were confirmed genuine** – forcing the  $C$ -optimal choice measurably costs more  $T$  in every one, none are ties. Worst single-state gap: exactly 1.0 extra expected query.

**Conclusion: these are different policies, not just different values.** A solver built to reach certainty fastest is not a substitute for one built to minimise the real scoring rule, at almost half of all decision points in this small a game.

## 6 Results at $n = 3$

Vocabulary (alphabet  $\{a, \dots, j\}$ ):  $W = \{\text{abc, cde, efg, ghi, ijb, bad, dec, fah}\}$ .

### 6.1 Optimal values, all ten excluded-letter scenarios

| excluded | $M_2$ hypotheses | $\mathbb{E}[C]$ | $\mathbb{E}[T]$ | mop-up tax |
|----------|------------------|-----------------|-----------------|------------|
| a        | 27,420           | 3.3700          | 4.0161          | 0.6461     |
| b        | 27,558           | 3.3060          | 3.9792          | 0.6733     |
| c        | 26,922           | 3.5938          | 4.2672          | 0.6734     |
| d        | 27,462           | 3.5741          | 4.2423          | 0.6682     |
| e        | 27,144           | 3.2555          | 3.8767          | 0.6212     |
| f        | 32,376           | 3.5607          | 4.1069          | 0.5463     |
| g        | 32,126           | 3.5576          | 4.1672          | 0.6096     |
| h        | 32,424           | 3.4869          | 4.1172          | 0.6302     |
| i        | 32,100           | 3.4886          | 4.1031          | 0.6145     |
| j        | 37,014           | 3.8287          | 4.4244          | 0.5956     |

The mop-up tax at  $n = 3$  (0.55–0.67 queries) is roughly 3–4 $\times$  the  $n = 2$  tax (0.18) – two data points, not yet a trend to extrapolate from, but consistent with ”more cells per line to mop up costs more.”

### 6.2 Policy divergence, sampled

Repeating Section 5’s comparison exhaustively at  $n = 3$  is not practical (millions of memoised states per scenario); a bounded random sample of 3000 states for excluded=‘a’ found 142 states where both solvers faced a genuine choice, of which **138 (97%) were confirmed genuine divergences**, none ties, worst gap again exactly 1.0 extra expected query. Consistent with, and if anything stronger than, the  $n = 2$  result.

### 6.3 Greedy vs. optimal

| excluded | $C$ -optimal | $C$ -greedy | $T$ -optimal | $T$ -greedy |
|----------|--------------|-------------|--------------|-------------|
| a        | 3.3700       | 5.1041      | 4.0161       | 8.3436      |
| b        | 3.3060       | 5.1876      | 3.9792       | 8.3441      |
| c        | 3.5938       | 5.3325      | 4.2672       | 8.3834      |
| d        | 3.5741       | 5.3069      | 4.2423       | 8.3925      |
| e        | 3.2555       | 5.1746      | 3.8767       | 8.3531      |
| f        | 3.5607       | 5.3824      | 4.1069       | 8.3001      |
| g        | 3.5576       | 5.3590      | 4.1672       | 8.2653      |
| h        | 3.4869       | 5.4239      | 4.1172       | 8.3020      |
| i        | 3.4886       | 5.4283      | 4.1031       | 8.3271      |
| j        | 3.8287       | 5.5378      | 4.4244       | 8.3437      |

Unlike at  $n = 2$ , greedy is now measurably worse than optimal for *both* objectives, not just  $T$ : roughly +1.7 queries for certainty, and consistently landing near 8.3 against a true optimum of 3.9–4.4 for the real objective – roughly **double** the optimal value, dramatically larger than the  $n = 2$  gap (2.27 vs. up to 3.45).  $n = 2$ ’s ”greedy matches optimal for certainty” coincidence (Section 5)

does not survive to  $n = 3$ : with more structure to exploit, one-step maximum-entropy selection is a substantially worse proxy for both objectives studied here, not a merely-occasionally-worse one.

## 7 Where exact solving stops being tractable

Two distinct obstacles, easy to conflate:

**1. Full-world enumeration has a factorial wall.** Computing  $V(H, R)$  exactly as defined requires the hypothesis space  $H$  to range over literal, fully-populated grids (so that every query’s outcome cell is well-defined for every hypothesis). The number of ways to permute the  $n^2 - n$  dross letters into the remaining cells is  $(n^2 - n - 1)!$ :

| $n$ | alphabet size | lines | dross permutations $(n^2 - n - 1)!$ |
|-----|---------------|-------|-------------------------------------|
| 2   | 5             | 6     | $2! = 2$                            |
| 3   | 10            | 8     | $6! = 720$                          |
| 4   | 17            | 10    | $12! = 479,001,600$                 |
| 5   | 26            | 12    | $20! \approx 2.43 \times 10^{18}$   |

$n = 4$  is already far beyond reach for literal enumeration. **No valid way around this is currently implemented.** A natural idea – represent a hypothesis by its  $(w, p)$  pair alone, without the dross permutation, since the permutation’s count is the same for every hypothesis and should cancel out of relative probabilities – was tried and found *not* to work for this objective: a dross letter’s actual disclosed cell depends on which specific permutation is realised, which is not determined by  $(w, p)$  alone, and the distribution over which cell it could be stops being uniform across hypotheses once some cells are already disclosed. A correct escape from full-world enumeration would need to properly count, for a given  $(w, p)$  and disclosure history  $R$ , how many consistent completions place a given untested letter at each possible cell – a real combinatorial counting problem, not yet solved.

**2. Even where full-world enumeration is possible, the search itself has a large branching factor.** Every query resolves to one of up to  $n^2$  distinct cells (not a small handful), so the reachable-state count under exhaustive expectimax search grows fast even at  $n = 3$  (3.4–4.5 million memoised states per scenario for MopUpSolver in Section 6, taking 45–80 seconds each on ordinary hardware). This is a separate obstacle from (1): it doesn’t go away even once  $H$  is confined to small, tractable vocabularies, and it will only get worse at  $n = 4$  (already-infeasible for reason (1)) and dramatically worse at  $n = 5$  (up to 21 distinct queryable letters, and a real dictionary’s vocabulary is  $\sim 10,000$  words rather than a handful).

$n = 5$  **at real-dictionary scale has not yet been validly attempted.** Given obstacle (1) already blocks full enumeration at  $n = 4$ , and no correct escape from it exists yet, there is currently no way to even *start* an exact computation at  $n = 5$ ’s real scale, independent of obstacle (2)’s branching-factor cost. This is stated as an open gap, not a measured wall – an earlier attempt at measuring this wall used a solver later found to be invalid for this objective, and no valid replacement measurement has been made.

## 8 Open questions

- 1. A correct pair-based (non-full-world) solver**, per Section 7(1) – the highest-value next step, since it is the only route past  $n = 4$ ’s factorial wall at all.
- 2. Branch-and-bound with a provable lower bound**, to reduce the cost from Section 7(2) without sacrificing exactness.

3. **Solving from mid-game states rather than the opening.** A real player starts from wherever they currently are, typically with  $|H|$  already reduced by prior disclosures; the opening state is the single largest and hardest one the game ever presents. Whether mid-game states are tractable further into  $n = 4$  or  $n = 5$  than the opening is, is worth checking directly rather than assuming either way.
4. **A variance-sensitive objective**, e.g. minimising  $\mathbb{E}[T^2]$  rather than  $\mathbb{E}[T]$  – requires tracking two moments through the recursion instead of one; not yet attempted.